STRUKDAT BAB 7

Tree Biner (Binary Tree)

---

1. Pengertian

Tree adalah struktur data non-linear yang terdiri dari node-node yang saling terhubung membentuk hubungan hierarki.

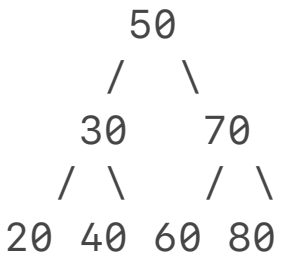
Binary Tree adalah tree yang setiap node memiliki maksimal:

```
2 anak
```

yaitu:

```
Left Child
Right Child
```

Contoh:



---

2. Istilah Penting Pada Tree

Root

Node paling atas.

```
50
```

Parent

Node yang memiliki anak.

```
50
```

adalah parent dari:

```
30 dan 70
```

Child

Node yang memiliki parent.

```
30
70
```

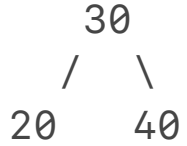
Leaf

Node tanpa anak.

```
20
40
60
80
```

Subtree

Bagian tree yang juga merupakan tree.



---

3. Struktur Node

```
struct Node{

    int data;

    struct Node* left;

    struct Node* right;

};
```

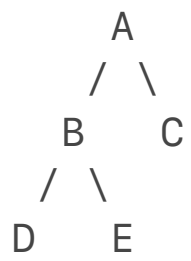
---

4. Membuat Node Baru

```
struct Node* createNode(int x){

    struct Node* node =
    (struct Node*)malloc(sizeof(struct Node));

    node->data = x;

    node->left = NULL;

    node->right = NULL;

    return node;

}
```

---

5. Representasi Binary Tree



Hubungan:

A -> root

B,C -> child A

D,E -> child B

---

6. Traversal Tree

Traversal adalah proses mengunjungi seluruh node.
Ada tiga traversal utama:

Preorder
Inorder
Postorder

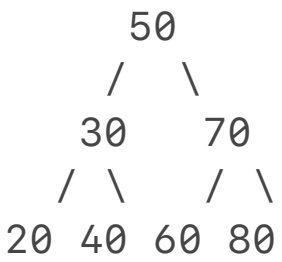
---

7. Preorder

Urutan:

Root
Left
Right

Contoh:



Hasil:

```
50 30 20 40 70 60 80
```

Implementasi:

```
void preorder(struct Node* root){

    if(root == NULL)
        return;

    printf("%d ",root->data);

    preorder(root->left);

    preorder(root->right);

}
```

---

8. Inorder

Urutan:

Left
Root
Right

Hasil:

20 30 40 50 60 70 80

Implementasi:

```
void inorder(struct Node* root){

    if(root == NULL)
        return;

    inorder(root->left);

    printf("%d ",root->data);

    inorder(root->right);

}
```

---

9. Postorder

Urutan:

Left
Right
Root

Hasil:

```
20 40 30 60 80 70 50
```

Implementasi:

```
void postorder(struct Node* root){

    if(root == NULL)
        return;

    postorder(root->left);

    postorder(root->right);

    printf("%d ",root->data);

}
```

---

10. Menghitung Jumlah Node

```
int countNode(struct Node* root){

    if(root == NULL)
        return 0;

    return 1
           + countNode(root->left)
           + countNode(root->right);
```

```
}
```

---

11. Menghitung Jumlah Leaf

```
int countLeaf(struct Node* root){

    if(root == NULL)
        return 0;

    if(root->left == NULL &&
       root->right == NULL)
        return 1;

    return countLeaf(root->left)
           +
           countLeaf(root->right);

}
```

---

12. Tinggi Tree

Tinggi tree:

```
Jumlah level maksimum
```

Implementasi:

```
int height(struct Node* root){

    if(root == NULL)
```

```
        return 0;

    int l = height(root->left);

    int r = height(root->right);

    return 1 + (l > r ? l : r);

}
```

---

13. Mencari Nilai Maksimum

```
int maximum(struct Node* root){

    if(root == NULL)
        return -1;

}
```

Dilakukan dengan traversal seluruh tree.

---

14. Mencari Nilai Minimum

Sama seperti maksimum.
Traversal seluruh node.

---

15. Binary Search Tree (BST)

BST adalah Binary Tree khusus.

Aturan:

```
Left < Root < Right
```

Contoh:

```
         50
        /  \
       30   70
      / \   / \
   20 40 60 80
```

---

16. Insert BST

Node baru dimasukkan sesuai aturan BST.

Implementasi:

```
struct Node* insert(struct Node* root,
                    int x){

    if(root == NULL){

        return createNode(x);

    }

    if(x < root->data){
```

```
        root->left =
        insert(root->left,x);

    }

    else{

        root->right =
        insert(root->right,x);

    }

    return root;

}
```

---

17. Search BST

```
int search(struct Node* root,
           int x){

    if(root == NULL)
        return 0;

    if(root->data == x)
        return 1;

    if(x < root->data)
        return search(root->left,x);

    return search(root->right,x);

}
```

---

18. Kompleksitas

Operasi	Binary Tree	BST
Traversal	O(n)	O(n)
Search	O(n)	O(log n)*
Insert	O(n)	O(log n)*
Count	O(n)	O(n)

`* jika tree seimbang`

---

19. Kesalahan Umum

Salah Traversal

Preorder:

```
Root Left Right
```

sering tertukar dengan:

```
Left Root Right
```

Tidak Mengecek NULL

Salah:

```
root->left
```

padahal:

```
root == NULL
```

Rekursi Tak Berhenti

Tidak membuat base case:

```
if(root == NULL)
```

BST Tidak Valid

Memasukkan data tanpa memperhatikan aturan:

```
Left < Root < Right
```

---

20. Implementasi Lengkap

```
#include <stdio.h>
#include <stdlib.h>

struct Node{

    int data;

    struct Node* left;
    struct Node* right;

};

struct Node* createNode(int x){

    struct Node* node =
    (struct Node*)malloc(sizeof(struct Node));

    node->data = x;

    node->left = NULL;
    node->right = NULL;
```

```
    return node;

}

struct Node* insert(struct Node* root,
                    int x){

    if(root == NULL)
        return createNode(x);

    if(x < root->data)
        root->left =
        insert(root->left,x);

    else
        root->right =
        insert(root->right,x);

    return root;

}

void inorder(struct Node* root){

    if(root == NULL)
        return;

    inorder(root->left);

    printf("%d ",root->data);

    inorder(root->right);

}

int main(){
```

```
    struct Node* root = NULL;

    root = insert(root,50);
    root = insert(root,30);
    root = insert(root,70);

    inorder(root);

    return 0;

}
```

___